Mentimeter

Agregare și moștenire

Mentimeter

Agregare și compunere

→ prezența unei referințe pentru un obiect într-o altă clasă

Mentimeter

Agregare (aggregation)

- weak association
- obiectul container poate să existe fără obiectele agregate
- ex: portofel fără bani

1 1

Mentimeter

Compunerea (composition)

- asociere puternică
- obiectul container NU poate să existe fără obiectele agregate
- obiectul container dispare atunci când dispar obiectele agregate
- ex: telefon fără procesor

Mentimeter

```
class Professor {}

class Student {
    Professor tutor = null;

    public Student(Professor professor) {
        this.tutor = professor;
    }
}

class Course {
    Professor professor;

    public Course(Professor professor) {
        this.professor = professor;
    }
}
```

2 1 1

Mentimeter

Inițializarea obiectelor conținute

- La definirea obiectului înainte de constructori
- În cadrul constructorului
- Chiar înainte de folosire (lazy initialization)

Mentimeter

```
class Page {
    private String content;
    public int numberOfPages;

    public Page(String content, int numberOfPages) {
        this.content       = content;
        this.numberOfPages = numberOfPages;
    }
}
```

Exemplu 0

1

Mentimeter

```java
class Book {
    private String title;
    private Page[] pages;
    private LibraryRow libraryRow = null;

    public Book(int size, String title, LibraryRow libraryRow) {
        this.libraryRow = libraryRow;
        this.title = title;

        pages = new Page[size];

        for (int i = 0; i < size; i++) {
            pages[i] = new Page( content: "Page " + i, i);
        }
    }
}
```

Exemplu 1

2 ? 2

Mentimeter

```
class LibraryRow {
    private String rowName = null;

    public LibraryRow(String rowName) {
        this.rowName = rowName;
    }
}
```

Exemplu 2

1

Mentimeter

```
class Library {

    public static void main(String[] args) {
        LibraryRow row = new LibraryRow( rowName: "a1");
        Book book = new Book( size: 100,  title: "title", row);
        book = null;
    }
}
```

Exemplu 3

2

Mentimeter

Moștenire (derivare)

- o clasă extinde o altă clasă
- preia funcționalitatea existentă și o modelează/adaugă alte funcționalități
- clasa extinsă = clasă părinte = clasă de bază = super clasă
- clasa care extinde = clasă copil = clasă derivată = sub clasă

Mentimeter

Java NU permite moștenire multiplă

- orice clasă are un părinte unic
- se rezolvă problema rombului

3

Mentimeter

Moștenire vs agregare

- moștenire - is a
- agregare - has a

Mentimeter

Când folosim agregare și nu moștenire?

Mentimeter

Clasa A extinde B

- moștenire: A moștenește toate metodele clasei B
- agregare: putem limita accesul numai la anumite metode
- e recomandat să folosim agregare dacă nu avem nevoie de toate metodele lui B

1 1

Mentimeter

Vrem acces direct asupra obiectelor agregate

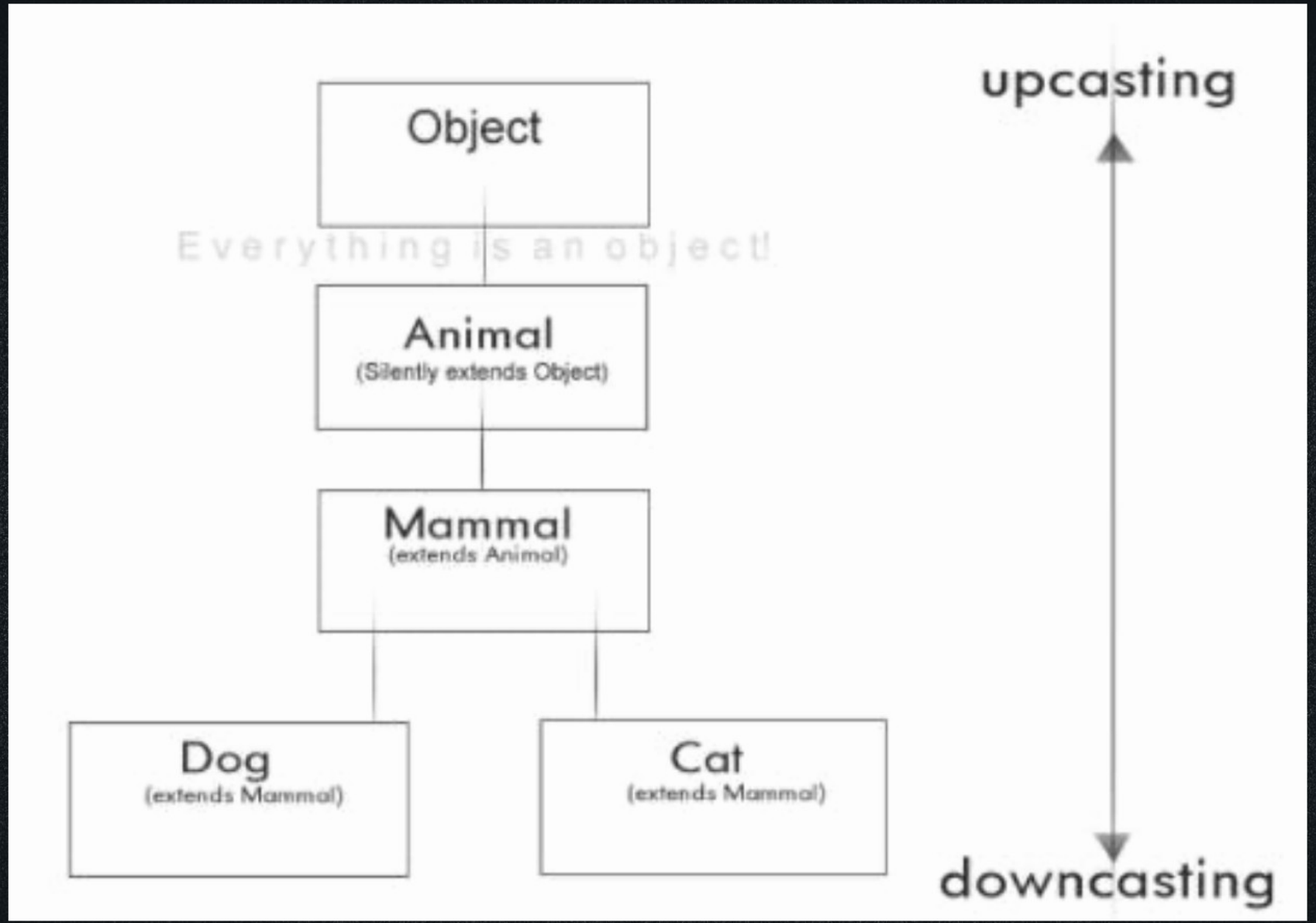
→ folosim agregare

Mentimeter

Când folosim moștenire și nu agregare?

→ când vrem să avem implementări mai specifice ale unei clase

Mentimeter



Upcasting & downcasting

2

Mentimeter

Upcasting

- cast de la clase specifice la clase mai generale
- se face automat
- ex: student.toString(), nu trebuie ((Object)student).toString()

Mentimeter

Downcasting

- cast de la clase generale la clase mai specifice
- NU se face automat
- generează excepție dacă obiectul nu are tipul la care facem downcasting
- cum aflăm tipul unui obiect? operatorul instanceof

1

Mentimeter

```java
class Course {}

class POO extends Course {
    void joinLab() {
        System.out.println("POO lab");
    }
}

class AA extends Course {
    void joinSeminar() {
        System.out.println("AA seminar");
    }
}
```

Exemplu clase

Mentimeter

```java
public class Main {
    public static void main(String[] args) {
        Course course1 = new POO();
        ((POO) course1).joinLab();

        Course course2 = new AA();
        ((AA) course2).joinSeminar();

        ((AA) course1).joinSeminar();
        ((POO) course2).joinLab();
    }
}
```

Așa no

2 ? 2

Mentimeter

```java
public class Main {
    public static void main(String[] args) {
        Course course1 = new POO();
        if (course1 instanceof POO) {
            ((POO) course1).joinLab();
        }

        Course course2 = new AA();
        if (course2 instanceof AA) {
            ((AA) course2).joinSeminar();
        }
    }
}
```

Așa da

2 1

Mentimeter

Chiar și așa instanceof NU e deloc recomandat

- cod repetitiv și greu de citit
- cod greu de menținut
- nu respectă principiul OCP (open for extension, closed for modification)

1

Mentimeter

Alternative

- polimorfism
- Visitor

Mentimeter

Polimorfism

- metode care depind de instanță (exact ce sub-clasă va avea la runtime)
- Suprascriere(overriding): funcție cu aceeași declarație, dar cu implementare diferită în copii vs părinte
- Supraîncărcare(overloading): funcție cu același nume, dar cu parametri diferiți

Mentimeter

```java
class Course {
    void join() {
        System.out.println("Join something");
    }

    void print() {
        System.out.println("Some course");
    }
}

class POO extends Course {
    void join() {
        System.out.println("Join POO lab");
    }

    void print(String professorName) {
        System.out.println("POO with " + professorName);
    }
}

class AA extends Course {
    void join() {
        System.out.println("Join AA seminar");
    }
}
```

Exemple metode

3 1

Mentimeter

```java
public class Main {
    public static void main(String[] args) {
        Course course = new POO();
        // Prints "Join POO lab", not "Join something"
        course.join();
    }
}
```

Exemplu apel

Mentimeter

```java
class Course {
    String name;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Course course = (Course) o;
        return Objects.equals(name, course.name);
    }
```

Exemplu suprascriere equals

1

Mentimeter

Implicații moștenire

- metode/câmpuri protected - au acces și clasele derivate
- metode/câmpuri private - nu are acces decât super clasa
- constructorii nu se moștenesc (dar pot fi apelați cu super)

Mentimeter

super

- instanța clasei părinte
- folosit pentru a apela o metodă suprascrisă
- sau pentru a apela constructorul părinte

Mentimeter

```java
class Course {
    String name;

    public Course(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "a course with the name " + name;
    }
}

class POO extends Course {
    public POO(String name) {
        super(name);
    }

    @Override
    public String toString() {
        return "POO: " + super.toString();
    }
}
```

Mentimeter

Demo

2